CSE 102T - Computer Programming II

Akdeniz University - Faculty of Engineering

Lab Assignment: Task Management System

Topics Covered: Interfaces, Generics, Abstract Classes, Inheritance, Comparable, Exceptions, Data Structures (List, Stack, Queue, Map, Set)

Estimated Time: 90 - 120 minutes

Files to Submit: All .java files listed in the File Structure section

1. Overview

In this lab, you will build a **Task Management System** that models different types of tasks (coding tasks and documentation tasks), processes them through a task processor, and manages them using various Java data structures. The system will demonstrate your understanding of interfaces, generics, abstract classes, inheritance, exception handling, and the Java Collections Framework.

You will implement a **Prioritizable** interface, an abstract **Task** class with two concrete subclasses, custom exception classes (both checked and unchecked), and a **TaskProcessor** class that operates on tasks using List, Stack, LinkedList (as Queue), Map, and Set.

2. File Structure

You will create the following Java files. The **TestTaskManager.java** file (main method) is provided for you.

File Name	Description
`Prioritizable.java`	Generic interface with getPriority() and compareTo()
`Task.java`	Abstract class implementing Prioritizable<Task>
`CodingTask.java`	Concrete subclass of Task
`DocumentationTask.java`	Concrete subclass of Task
`InvalidTaskException.java`	Checked exception (extends Exception)
`TaskOverflowException.java`	Unchecked exception (extends RuntimeException)
`TaskProcessor.java`	Processes tasks using multiple data structures
`TestTaskManager.java`	PROVIDED - Main method with all test cases

3. Part 1 - Prioritizable Interface

Create a **generic interface** called `Prioritizable<T>` that defines the contract for any object that can be prioritized and compared.

3.1 What is an Interface?

An interface in Java is a reference type that contains **only abstract methods** and **constants**. It defines a contract: any class that implements the interface **must** provide concrete implementations of all its abstract methods. Interfaces allow unrelated classes to share common behavior.

3.2 Requirements

- The interface must be **generic** with type parameter <T>
- It must **extend** `Comparable<T>`
- Declare one abstract method: `int getPriority()`

3.3 How Comparable Works

When an interface extends `Comparable<T>`, any class implementing your interface must also implement the `compareTo(T other)` method. The compareTo method must return:

- A **negative** integer if this object is "less than" the other
- **Zero** if they are "equal"
- A **positive** integer if this object is "greater than" the other

3.4 Template

```
public interface Prioritizable<T> extends Comparable<T> {
    // Declare: int getPriority();
}
```

Note: By extending Comparable<T>, you do NOT need to redeclare compareTo here. The implementing class will provide it.

4. Part 2 - Abstract Task Class

Create an **abstract class** called `Task` that implements `Prioritizable<Task>`. This class holds the common fields and behavior shared by all task types.

4.1 What is an Abstract Class?

An abstract class is a class declared with the `abstract` keyword. It **cannot be instantiated** directly. It can contain both abstract methods (no body) and concrete methods (with body). Subclasses **must** implement all abstract methods.

4.2 Fields (all private)

Type	Name	Description
`String`	`id`	Unique task identifier (e.g., "T001")
`String`	`title`	Short title of the task
`int`	`priority`	Priority level (1 = highest, 5 = lowest)
`boolean`	`completed`	Whether the task is done (default: false)

4.3 Constructor

```
Task(String id, String title, int priority) throws InvalidTaskException
```

The constructor must **validate** the priority. If priority is less than `1` or greater than `5`, throw an `InvalidTaskException` with the message:
`"Invalid priority: [value]. Must be between 1 and 5."`

If valid, assign the fields. Set completed to false by default.

4.4 Methods to Implement

Method Signature	Description
`int getPriority()`	Returns the priority field (from Prioritizable interface)
`int compareTo(Task other)`	Compare by priority: this.priority - other.priority (lower number = higher priority = comes first)
`boolean equals(Object obj)`	Two tasks are equal if they have the same id. Use instanceof check, then cast and compare.
`String toString()`	Return: "[id] title (Priority: priority)"
`void markCompleted()`	Set completed to true
`abstract String getCategory()`	Abstract - subclasses will return their category name

4.5 How to Override equals()

When overriding equals, you must follow this pattern:

1. Check if the parameter `obj` is an instance of your class using `instanceof`
2. If not, return `false`
3. If yes, cast `obj` to your class type
4. Compare the relevant field(s) - in this case, compare the `id` fields using `.equals()` for Strings

```
@Override
public boolean equals(Object obj) {
    if (obj instanceof Task) {
        Task other = (Task) obj;
        return this.id.equals(other.id);
    }
    return false;
}
```

4.6 How to Override compareTo()

The `compareTo` method defines the natural ordering. For tasks, we order by priority number. Since priority 1 is the most urgent, a simple subtraction works:

```
@Override
public int compareTo(Task other) {
    return this.priority - other.priority;
}
```

Hint: This means a task with priority 1 will come before a task with priority 3 when sorted.

4.7 Getter Methods

Provide standard getter methods for all four fields: `getId()`, `getTitle()`, `getPriority()`, `isCompleted()`.

5. Part 3 - Concrete Subclasses

Create two classes that **extend** the abstract `Task` class. Each subclass adds its own unique field and implements the abstract `getCategory()` method.

5.1 CodingTask

Extra Field	Type	Description
`programmingLanguage`	`String`	The language used (e.g., "Java", "Python")

- **Constructor:** `CodingTask(String id, String title, int priority, String programmingLanguage) throws InvalidTaskException` - Call the parent constructor with `super(id, title, priority)`, then set the extra field.
- **getCategory():** Return `"Coding"`
- **getLanguage():** Return the programming language
- **toString():** Override to return `"[id] title (Priority: priority) [Coding - language]"`

5.2 DocumentationTask

Extra Field	Type	Description
`pageCount`	`int`	Number of pages in the document

- **Constructor:** `DocumentationTask(String id, String title, int priority, int pageCount) throws InvalidTaskException` - Call super, then set the extra field.
- **getCategory():** Return `"Documentation"`
- **getPageCount():** Return the page count
- **toString():** Override to return `"[id] title (Priority: priority) [Docs - pageCount pages]"`

  Important: Both constructors propagate InvalidTaskException from the parent. The throws clause must be declared in the method signature.

6. Part 4 - Custom Exceptions

You will create **two** custom exception classes to understand the difference between **checked exceptions** and **unchecked (runtime) exceptions**.

6.1 Checked vs. Unchecked Exceptions

Checked exceptions extend `Exception`. The compiler forces you to either catch them with try-catch or declare them with `throws`. They represent recoverable conditions (e.g., invalid input that can be corrected).

Unchecked exceptions extend `RuntimeException`. The compiler does NOT force you to handle them. They represent programming errors or unrecoverable conditions (e.g., exceeding a capacity limit).

6.2 InvalidTaskException (Checked)

- Extends `Exception`
- Constructor takes a `String message` and passes it to `super(message)`

```
public class InvalidTaskException extends Exception {
    public InvalidTaskException(String message) {
        super(message);
    }
}
```

6.3 TaskOverflowException (Unchecked)

- Extends `RuntimeException`
- Constructor takes a `String message` and passes it to `super(message)`

```
public class TaskOverflowException extends RuntimeException {
    public TaskOverflowException(String message) {
        super(message);
    }
}
```

Notice: InvalidTaskException requires try-catch or throws at the call site. TaskOverflowException does NOT require explicit handling, but you can still catch it if you want.

7. Part 5 - TaskProcessor Class

This is the core class that manages tasks using multiple data structures. It demonstrates practical usage of `List`, `Stack`, `LinkedList` (as Queue), `Map`, and `Set`.

7.1 Fields

Type	Name	Purpose
`ArrayList<Task>`	`taskList`	Stores all tasks in insertion order
`Stack<Task>`	`undoStack`	Tracks recently completed tasks for undo
`LinkedList<Task>`	`taskQueue`	FIFO queue for processing tasks in order
`HashMap<String, Task>`	`taskMap`	Fast lookup of tasks by their id
`HashSet<String>`	`categories`	Tracks unique task categories
`int`	`maxCapacity`	Maximum number of tasks allowed

7.2 Constructor

`TaskProcessor(int maxCapacity)` - Initialize all data structure fields (create new empty instances) and set the maxCapacity.

7.3 Required Imports

```
import java.util.ArrayList;
import java.util.Stack;
import java.util.LinkedList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.Set;
```

7.4 Methods

7.4.1 addTask(Task task)

Adds a task to the system. Must perform these checks in order:

- If `taskList.size() >= maxCapacity`, throw a `TaskOverflowException` with message `"Task limit reached: [maxCapacity]"`
- If the task's id already exists in `taskMap`, throw `IllegalArgumentException` with message `"Duplicate task ID: [id]"`
- If valid, add the task to: `taskList`, `taskQueue` (add to end), `taskMap` (key=id, value=task), and `categories` (add the task's category)

7.4.2 processNextTask()

Removes and returns the **first** task from `taskQueue` (FIFO order). Marks the task as completed, pushes it onto `undoStack`, and returns it. If the queue is empty, return `null`.

> *Hint: Use LinkedList's poll() method to remove from front. poll() returns null if empty.*

7.4.3 undoLastProcess()

Pops the most recently processed task from `undoStack` and returns it. If the stack is empty, return `null`.

Hint: Use Stack's isEmpty() to check, then pop() to remove.

7.4.4 findTaskById(String id)

Uses `taskMap` to look up and return a task by its id. Returns `null` if not found.

Hint: Use HashMap's get() method. It returns null if the key doesn't exist.

7.4.5 getSortedTasks()

Returns a **new** `ArrayList<Task>` containing all tasks from `taskList`, sorted by priority (ascending). Do **NOT** modify the original `taskList`.

```
ArrayList<Task> sorted = new ArrayList<>(taskList); // copy
Collections.sort(sorted); // uses compareTo
return sorted;
```

Note: Collections.sort() works because Task implements Comparable<Task> through the Prioritizable interface.

7.4.6 getCategories()

Returns the `categories` Set. This shows all unique categories that have been added.

7.4.7 Getter Methods

- `List<Task> getTaskList()` - returns taskList
- `int getQueueSize()` - returns taskQueue.size()
- `int getStackSize()` - returns undoStack.size()

8. Provided Test Code - TestTaskManager.java

The following main method is **provided for you**. Copy it exactly into `TestTaskManager.java`. Your implementation must produce the expected output shown in Section 9.

WARNING: Do NOT modify this test code. Your classes must work with it exactly as written.

```
import java.util.List;

public class TestTaskManager {
public static void main(String[] args) {

System.out.println("=== TASK MANAGEMENT SYSTEM TEST ===");
System.out.println();

// --- Test 1: Create tasks ---
System.out.println("--- Test 1: Creating Tasks ---");
try {
CodingTask ct1 = new CodingTask("T001", "Build Login API", 2, "Java");
CodingTask ct2 = new CodingTask("T002", "Fix Payment Bug", 1, "Python");
DocumentationTask dt1 = new DocumentationTask("T003", "Write User Guide", 3, 45);
DocumentationTask dt2 = new DocumentationTask("T004", "API Reference", 1, 120);
CodingTask ct3 = new CodingTask("T005", "Database Migration", 4, "Java");

System.out.println(ct1);
System.out.println(ct2);
System.out.println(dt1);
System.out.println(dt2);
System.out.println(ct3);

// --- Test 2: Invalid task ---
System.out.println();
System.out.println("--- Test 2: Invalid Task (Checked Exception) ---");
try {
CodingTask bad = new CodingTask("T099", "Bad Task", 7, "Go");
} catch (InvalidTaskException e) {
System.out.println("Caught: " + e.getMessage());
}

// --- Test 3: TaskProcessor ---
System.out.println();
System.out.println("--- Test 3: Adding Tasks to Processor ---");
TaskProcessor processor = new TaskProcessor(5);
processor.addTask(ct1);
processor.addTask(ct2);
processor.addTask(dt1);
processor.addTask(dt2);
processor.addTask(ct3);
System.out.println("Tasks added: " + processor.getTaskList().size());
System.out.println("Queue size: " + processor.getQueueSize());

// --- Test 4: Overflow (Runtime Exception) ---
System.out.println();
System.out.println("--- Test 4: Task Overflow (Runtime Exception) ---");
try {
CodingTask extra = new CodingTask("T006", "Extra Task", 2, "C++");
processor.addTask(extra);
} catch (TaskOverflowException e) {
System.out.println("Caught: " + e.getMessage());
}

// --- Test 5: Duplicate ID ---
System.out.println();
System.out.println("--- Test 5: Duplicate Task ID ---");
try {
CodingTask dup = new CodingTask("T001", "Duplicate", 2, "Ruby");
```

```
processor.addTask(dup);
} catch (IllegalArgumentException e) {
System.out.println("Caught: " + e.getMessage());
}

// --- Test 6: Sorted tasks ---
System.out.println();
System.out.println("--- Test 6: Tasks Sorted by Priority ---");
List<Task> sorted = processor.getSortedTasks();
for (Task t : sorted) {
System.out.println(" " + t);
}

// --- Test 7: Process queue ---
System.out.println();
System.out.println("--- Test 7: Processing Queue (FIFO) ---");
Task processed1 = processor.processNextTask();
Task processed2 = processor.processNextTask();
System.out.println("Processed: " + processed1.getId());
System.out.println("Processed: " + processed2.getId());
System.out.println("Remaining in queue: " + processor.getQueueSize());

// --- Test 8: Undo ---
System.out.println();
System.out.println("--- Test 8: Undo Last Process ---");
Task undone = processor.undoLastProcess();
System.out.println("Undone: " + undone.getId());
System.out.println("Stack size after undo: " + processor.getStackSize());

// --- Test 9: Find by ID ---
System.out.println();
System.out.println("--- Test 9: Find Task by ID (Map) ---");
Task found = processor.findTaskById("T003");
System.out.println("Found: " + found);
Task notFound = processor.findTaskById("T999");
System.out.println("Not found: " + notFound);

// --- Test 10: Categories ---
System.out.println();
System.out.println("--- Test 10: Unique Categories (Set) ---");
System.out.println("Categories: " + processor.getCategories());

// --- Test 11: equals ---
System.out.println();
System.out.println("--- Test 11: Task Equality ---");
CodingTask eqTest = new CodingTask("T001", "Different Name", 5, "Rust");
System.out.println("T001 equals T001 (diff name): " + ct1.equals(eqTest));
System.out.println("T001 equals T002: " + ct1.equals(ct2));

} catch (InvalidTaskException e) {
System.out.println("Unexpected error: " + e.getMessage());
}

System.out.println();
System.out.println("=== ALL TESTS COMPLETE ===");
}
}
```

9. Expected Output

When you run TestTaskManager, your output should match the following:

```
=== TASK MANAGEMENT SYSTEM TEST ===

--- Test 1: Creating Tasks ---
[T001] Build Login API (Priority: 2) [Coding - Java]
[T002] Fix Payment Bug (Priority: 1) [Coding - Python]
[T003] Write User Guide (Priority: 3) [Docs - 45 pages]
[T004] API Reference (Priority: 1) [Docs - 120 pages]
[T005] Database Migration (Priority: 4) [Coding - Java]

--- Test 2: Invalid Task (Checked Exception) ---
Caught: Invalid priority: 7. Must be between 1 and 5.

--- Test 3: Adding Tasks to Processor ---
Tasks added: 5
Queue size: 5

--- Test 4: Task Overflow (Runtime Exception) ---
Caught: Task limit reached: 5

--- Test 5: Duplicate Task ID ---
Caught: Duplicate task ID: T001

--- Test 6: Tasks Sorted by Priority ---
[T002] Fix Payment Bug (Priority: 1) [Coding - Python]
[T004] API Reference (Priority: 1) [Docs - 120 pages]
[T001] Build Login API (Priority: 2) [Coding - Java]
[T003] Write User Guide (Priority: 3) [Docs - 45 pages]
[T005] Database Migration (Priority: 4) [Coding - Java]

--- Test 7: Processing Queue (FIFO) ---
Processed: T001
Processed: T002
Remaining in queue: 3

--- Test 8: Undo Last Process ---
Undone: T002
Stack size after undo: 1

--- Test 9: Find Task by ID (Map) ---
Found: [T003] Write User Guide (Priority: 3) [Docs - 45 pages]
Not found: null

--- Test 10: Unique Categories (Set) ---
Categories: [Coding, Documentation]

--- Test 11: Task Equality ---
T001 equals T001 (diff name): true
T001 equals T002: false

=== ALL TESTS COMPLETE ===
```

Note: The order of elements in Set output (Test 10) may vary. Both [Coding, Documentation] and [Documentation, Coding] are acceptable because Sets are unordered. Similarly, for Test 6, tasks with the same priority (T002 and T004 both have priority 1) may appear in either order.

10. Implementation Tips

10.1 Suggested Implementation Order

1. Start with the two exception classes (simplest)
2. Create the Prioritizable interface
3. Implement the abstract Task class
4. Implement CodingTask and DocumentationTask
5. Build TaskProcessor method by method, testing as you go

10.2 Common Mistakes to Avoid

- Forgetting `throws InvalidTaskException` on constructors that call `super()` which throws
- Using == instead of `.equals()` when comparing Strings in the equals method
- Modifying the original list in getSortedTasks() instead of creating a copy
- Forgetting to add the task to ALL four data structures in addTask()
- Not calling `markCompleted()` inside processNextTask()

10.3 Data Structure Quick Reference

Structure	Key Methods Used	Why Used Here
ArrayList	`add(), size(), get()`	Ordered collection, supports sorting
Stack	`push(), pop(), isEmpty()`	LIFO - last completed = first to undo
LinkedList (as Queue)	`add(), poll(), size()`	FIFO - process in insertion order
HashMap	`put(), get(), containsKey()`	O(1) lookup by task ID
HashSet	`add()`	Track unique categories only

11. Grading Rubric

Component	Points
Prioritizable interface (generic, extends Comparable)	10
Task abstract class (fields, constructor, methods, validation)	20
CodingTask and DocumentationTask (inheritance, toString)	15
InvalidTaskException (checked) + TaskOverflowException (unchecked)	10
TaskProcessor - addTask with overflow + duplicate check	15
TaskProcessor - processNextTask + undoLastProcess	10
TaskProcessor - findTaskById + getSortedTasks + getCategories	10
Code compiles and produces correct output	10
Total	**100**

Good luck! Remember: compile often, test each class individually, and read error messages carefully.